



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Registers and Arithmetic

CMSC 131 Bootcamp Block 2

Prepared by: Rene Andre Bedonia Jocsing

Published: August 26, 2026

Deadline: August 26, 2026

Prerequisites

One archive holds everything this block needs, including a project that already builds. You can start today without Git, without a GitHub account, and without last session's folder.

You need this [archive](#). Unzip it somewhere permanent and work inside the folder it makes.

Session Objectives

- Name the general-purpose registers and say what each one is conventionally for
- Read an integer from the user and print one back using Carter's I/O routines
- Perform addition, subtraction, multiplication, and division on registers
- Predict what integer division does to a remainder
- Build a working temperature converter from an empty file

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, b1 through b7, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

You need the tools from Block 1 installed, meaning NASM, a 32-bit `gcc`, `make`, and a `bash` to type in.

The archive above is a finished version of Block 1's project with today's files already in it, so unzipping it is all the setup this block needs. Check that it works before you start:

```
1 make check
```

OK: `skel matches skel.expected` means you're ready. If you built your own `pc_asm` folder in Block 1 and would prefer to keep working in it, copy five files across instead. Four of them are below. The fifth is `asm_io.asm`, which the archive carries a modified copy of and yours doesn't. Every check from today onward depends on that modification. "Checking it" at the end of Part 5 says what it is.

File	What it's for
b2_starter.asm	The exercise with the prompts written and the arithmetic left to you
convert.input	One temperature, fed to your program by <code>make check</code>
convert.expected	What a correct program prints for that temperature
b2_validation.py	Prints the right answers for the other cases, so you can check them by hand

Ninety minutes, divided roughly into twenty on registers and the echo program, twenty on arithmetic, forty on the converter, and ten on checking it.

Part 1: What a Register Is

A **register** is a storage slot inside the processor itself, wired directly to the arithmetic unit.

Eight of them you can use freely in 32-bit x86, each holding 32 bits:

```
1  eax ebx ecx edx esi edi esp ebp
```

That's the entire set. A Python program can have ten thousand variables. Your assembly program has eight slots. Two of them (`esp` and `ebp`) are spoken for by a region of memory called the stack, which Block 7 gets to. So really you have six.

This shortage is the single biggest difference between assembly and everything you've written before. You'll spend effort deciding what lives in a register and what gets pushed aside to make room. That isn't a flaw in the language. It's what the hardware is.

The names are historical. The processor mostly doesn't care which one you use, though some instructions have opinions, and other programmers have expectations:

Register	Conventional use
<code>eax</code>	Accumulator. Arithmetic results land here, and <code>mul/div</code> require it
<code>ebx</code>	Base. A general-purpose scratch register
<code>ecx</code>	Counter. Loop instructions decrement it automatically
<code>edx</code>	Data. Holds the overflow half of a multiply, and the remainder of a divide
<code>esi</code>	Source index. The "read from here" pointer in string operations
<code>edi</code>	Destination index. The "write to here" pointer

Follow the conventions. When Block 3 makes `div` clobber `edx` whether you wanted it to or not, you'll already have expected it.

Part 2: Getting Numbers in and Out

Carter's library gives you six routines. They all communicate through `eax`.

Routine	What it does
read_int	Reads an integer from the keyboard into <code>eax</code>
print_int	Prints the integer in <code>eax</code> , read as signed
print_string	Prints the null-terminated string whose address is in <code>eax</code>
print_char	Prints the single character in <code>eax</code>
read_char	Reads one character into <code>eax</code>
print_nl	Prints a newline

This is the whole set. Later blocks refer back to this table rather than repeating it. `print_char` looks useless next to `print_string` until Block 5 asks you to put spaces between numbers, at which point declaring a string for one character starts to feel silly.

There's one more thing to know about all six. It saves you a class of bug that's miserable to find. **They give you back every register you had.** Each one saves the lot on entry and restores them before returning, so a value you left in `ebx` is still there afterwards. The single exception is `eax`, which is how you hand a value in and how a value comes back.

That's a promise Carter's routines make, and not a promise the hardware makes. Routines you write yourself keep nothing safe unless you write the code that saves it. Block 7 shows you how.

Start from `skel.asm`, renamed:

```
1 cp skel.asm convert.asm
```

Then a first program that echoes a number back:

```
1  %include "asm_io.inc"
2
3  segment .data
4  prompt      db "Enter a number: ", 0
5  answer_msg  db "You typed: ", 0
6
7  segment .bss
8
9  segment .text
10     global _asm_main
11     _asm_main:
12         enter    0,0
13         pusha
14
15         mov     eax, prompt
16         call    print_string
17         call    read_int          ; eax now holds what the user typed
18         mov     ebx, eax         ; stash it, because print_string needs eax
19
20         mov     eax, answer_msg
21         call    print_string
22         mov     eax, ebx         ; bring the number back
23         call    print_int
24         call    print_nl
25
26         popa
27         mov     eax, 0
28         leave
29         ret
```

Build and run it:

```
1 make PROG=convert run
```

Why does the number get stashed in `ebx`? Look at the line `mov ebx, eax` again. It seems pointless, since the number is already in `eax`.

The problem is that `print_string` also needs `eax` for the address of the string. The moment you load `answer_msg` into `eax`, the number the user typed is gone. There's no undo.

So you move it somewhere safe first. This is the register shortage from Part 1 showing up in the first program you write. It'll keep showing up. Before you overwrite a register, ask whether anything still needs what's in it.

Watch the brackets here. `mov eax, prompt` loads the *address* of `prompt`. Writing `mov eax, [prompt]` instead loads the first four bytes of the text itself, printing garbage. `print_string` wants an address. Block 7 explains why the brackets mean what they mean.

Part 3: Arithmetic

Addition and subtraction are direct:

```
1 add    eax, ebx        ; eax = eax + ebx
2 sub    eax, 32         ; eax = eax - 32
```

Multiplication and division aren't. They're where the conventions from Part 1 start to bite.

```
1 mov    eax, 9
2 mov    ebx, 5
3 mul    ebx             ; edx:eax = eax * ebx
```

`mul` always multiplies whatever is in `eax` by the operand you name, and writes the answer across **two** registers, `edx` and `eax` together. For the small numbers in this block, `edx` will be zero and the whole answer fits in `eax`. Block 3 covers what happens when it doesn't.

Division is fussier:

```
1 mov    edx, 0          ; REQUIRED: clear the high half first
2 mov    eax, 100
3 mov    ebx, 7
4 div    ebx             ; eax = quotient (14), edx = remainder (2)
```

Why must `edx` be cleared before `div`? Because `div` doesn't divide `eax` by the operand alone. It divides the 64-bit value formed by `edx:eax`, the two registers glued together, and puts the quotient in `eax` and the remainder in `edx`.

So whatever junk is sitting in `edx` becomes part of your dividend. If `edx` happens to hold 5 and `eax` holds 100, you haven't asked for $100 \div 7$. You've asked for $21474836580 \div 7$. That doesn't fit in `eax`, so the processor raises a divide error and your program dies.

`mov edx, 0` before every `div` isn't superstition. Write it every time.

Part 4: Integer Division Truncates

No fractions here. `div` gives you a whole-number quotient and a separate remainder. If you ignore the remainder it's gone.

```
1 100 / 7 ->  eax = 14, edx = 2      (not 14.2857...)
2   6 / 10 ->  eax = 0,  edx = 6      (not 0.6)
```

Look hard at that second line. Dividing before multiplying throws the answer away.

Part 5: The Temperature Converter

Now put it together. Either keep working in the `convert.asm` you made above, or start from `b2_starter.asm`, which has the four messages declared and the arithmetic left blank.

```
1 cp b2_starter.asm convert.asm
```

Your program should:

1. Prompt for a temperature in degrees Celsius.
2. Convert it to Fahrenheit and print the result.
3. Convert that Fahrenheit value to Kelvin and print the result.
4. Convert that Kelvin value back to Celsius and print the result.

The formulas:

- Celsius to Fahrenheit: $F = (C * 9) / 5 + 32$
- Fahrenheit to Kelvin: $K = ((F - 32) * 5) / 9 + 273$
- Kelvin to Celsius: $C = K - 273$

Rules for this exercise. Use whole-number arithmetic only, no floating point and no FPU. Use the I/O routines from Part 2. Assume the input is sensible, so no negative temperatures and nothing absurd.

Expected output:

```
1 Input a temperature in Celsius: 0
2 The temperature in Fahrenheit from Celsius is: 32
3 The temperature in Kelvin from Fahrenheit is: 273
4 The temperature in Celsius from Kelvin is: 0
5
6 Input a temperature in Celsius: 69
7 The temperature in Fahrenheit from Celsius is: 156
8 The temperature in Kelvin from Fahrenheit is: 341
9 The temperature in Celsius from Kelvin is: 68
10
11 Input a temperature in Celsius: 100
12 The temperature in Fahrenheit from Celsius is: 212
13 The temperature in Kelvin from Fahrenheit is: 373
14 The temperature in Celsius from Kelvin is: 100
```

Feed it 69 and you get 68 back. Feed it 100 and you get 100. Why does one round-trip lose a degree and the other doesn't?

Run `python b2_validation.py` to see every case from 0 to 100 worked out the same way your program has to work them out. Eighty of those hundred and one round trips don't come back to where they started. None of that is a bug.

On Linux the command is `python3 b2_validation.py`. Ubuntu ships no plain python, so the shorter name answers `command not found` there.

Checking it

```
1 make PROG=convert check
```

Success prints OK: `convert` matches `convert.expected`.

That isn't the run you've been doing by hand. `check` feeds your program `convert.input`, a file holding the single line 69, so it reads from a file rather than from you. Ask for the same run with the comparison left off and you can see what that does:

```
1 make PROG=convert replay
```

Four lines come back. If `make` had to rebuild first, its build commands sit above them, so read from the bottom:

```
1 Input a temperature in Celsius: 69
2 The temperature in Fahrenheit from Celsius is: 156
3 The temperature in Kelvin from Fahrenheit is: 341
4 The temperature in Celsius from Kelvin is: 68
```

That's `convert.expected`, byte for byte, which is why the check passed. It also reads like the run you did by hand, 69 and all. That part took arranging.

When you type at a terminal, the terminal prints your keystrokes back at you as they arrive. That `echo` is what puts 69 on screen after the prompt, and your Enter is what moves the next line down. Your program printed neither one. Take the keyboard away and the `echo` goes too, so a program reading from a file would leave the prompt sharing a line with the Fahrenheit result, with the number it read nowhere on screen.

So `read_int` does the echoing itself when there's no terminal to do it. It asks whether its input is a keyboard. When the answer is no, it prints the number it just read, followed by the line break your Enter would have caused. That's why `convert.expected` is readable, and why one file describes both runs.

A routine called `read_int` that prints is odd. It isn't Carter's doing either. This course added the `echo` to `asm_io.asm` and marked it in place, so you can go and read it.

Written as $(C * 9) / 5$, the formula works. Written as $(C / 5) * 9$, it doesn't.

Take $C = 69$. Multiplying first gives 621, then dividing by 5 gives 124, and adding 32 gives 156. Dividing first gives $69 \div 5 = 13$ with the remainder discarded, then $13 \times 9 = 117$, then 149. You're seven degrees off. Nothing in the program will tell you.

The general rule applies well beyond this exercise. In integer arithmetic, do every multiplication you can before the first division. Each division you perform early throws away precision you can't get back.

Hints, in Increasing Order of Spoiler

Read one, go back to your code, and only come back for the next if you're still stuck.

1. Work out which values have to survive at the same time, and which register each one lives in, before you write a single instruction. Write that list in a comment at the top. Celsius has to survive until you compute Fahrenheit, and Fahrenheit has to survive until you compute Kelvin and until you print it.
2. `mul` and `div` take `eax` and `edx` whether you offered them or not, so those two are scratch. `esi` and `edi` are the natural homes for values that have to live across the arithmetic, since nothing in this program has an opinion about them.
3. Each conversion is the same six lines. Load the source value into `eax`, put the multiplier in another register, `mul`, clear `edx`, put the divisor in a register, `div`. Then add the constant and move the result

out of `eax` before you touch `eax` again. If your second conversion looks nothing like your first, one of them is wrong.

Testing Checklist

Core Functionality

- 0 °C produces 32 °F, 273 K, and 0 °C
- 100 °C produces 212 °F, 373 K, and 100 °C
- 69 °C produces 156 °F, 341 K, and 68 °C
- Room temperature, around 20 to 25 °C, produces plausible values
- Larger inputs such as 200 and 500 don't crash
- `make PROG=convert check` prints OK: `convert` matches `convert.expected`

Common Pitfalls

- Dividing before multiplying, which silently produces wrong answers
- Forgetting `mov edx, 0` before a `div`, which crashes or produces nonsense
- Overwriting a value with `print_string` before printing it
- Expecting a fractional result from `div`, which never happens
- Reusing `eax` for the next conversion before you finished with the last one

Architecture Review

What We Built

- A program that reads input, computes three chained conversions, and reports each
- A working habit of stashing values before a call clobbers them
- Whole-number arithmetic that respects the order of operations

Key Takeaways

1. **Six usable registers.** Deciding what occupies them is most of the work.
2. **`mul` and `div` reach for `eax` and `edx` whether you asked or not.** Plan around it.
3. **Always clear `edx` before `div`.** The dividend is 64 bits wide, not 32.
4. **Multiply first, divide last.** Early division discards precision permanently.
5. **A lost degree in a round-trip is truncation, not a bug.** Knowing the difference is the point.

Next Session Preview

- How a pattern of bits becomes a positive or negative number
- Two's complement, and why subtraction is addition in disguise
- `div` against `idiv`, and what changes when the sign matters
- Multiplications too large for one register